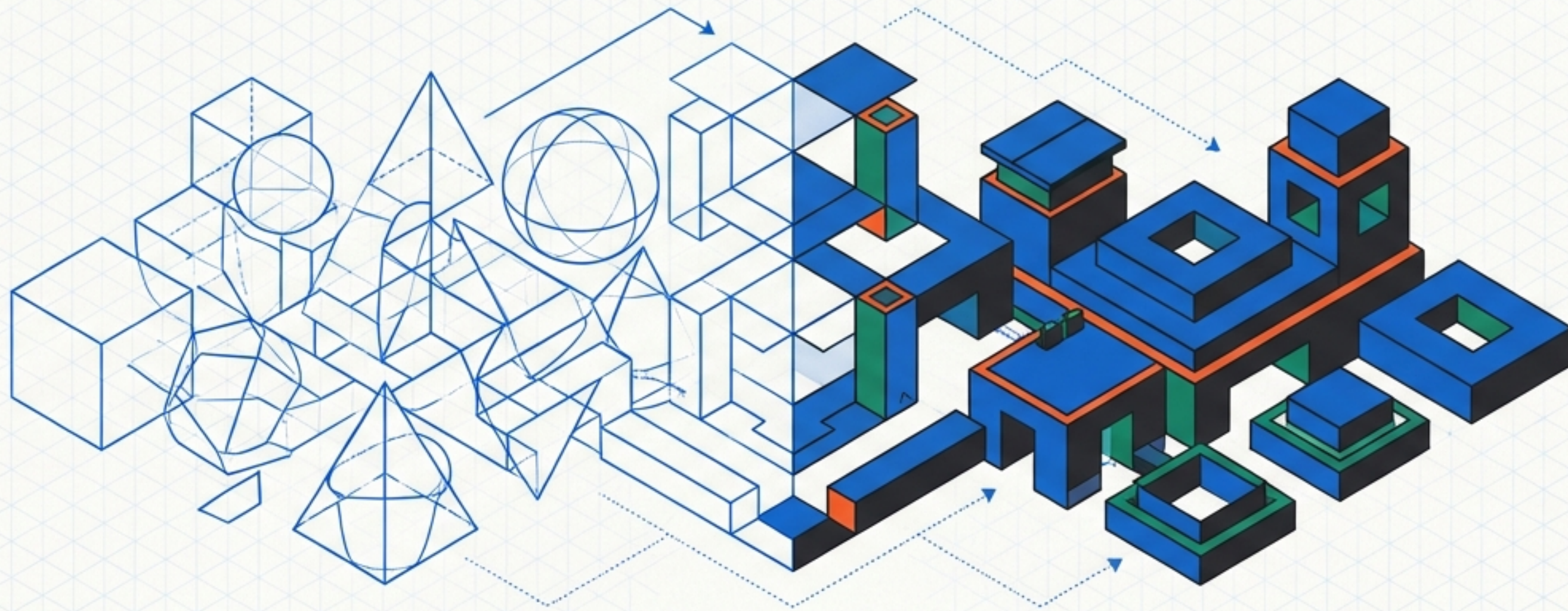


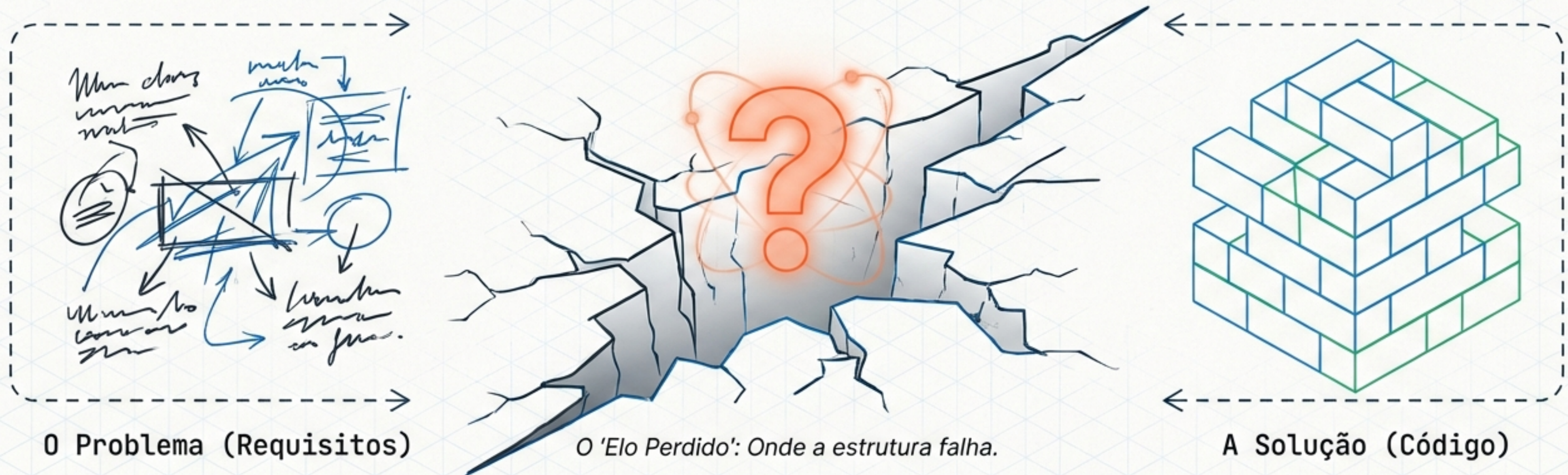


Fundamentos do Design de Software

Da Abstração à Implementação

A Lacuna Perigosa

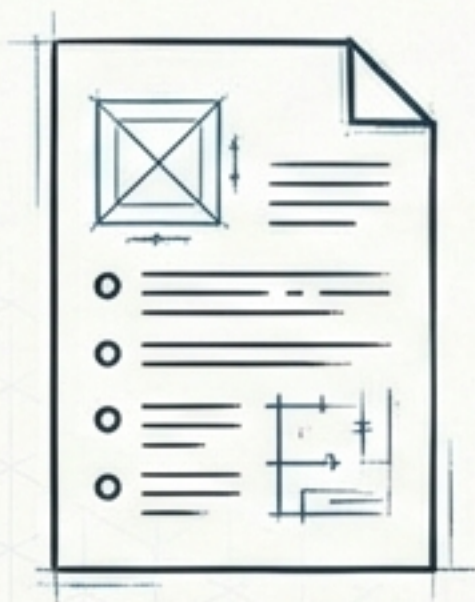
No desenvolvimento de sistemas, a transição direta da **Elicitação de Requisitos** para a **Implementação** resulta frequentemente em anomalias estruturais e dívida técnica.



Insight: O desenvolvimento transcende a mera escrita de código-fonte.

O Que é Design de Software?

É o processo de engenharia onde os requisitos são traduzidos em uma abstração que serve de **diretriz inquestionável** para a codificação.



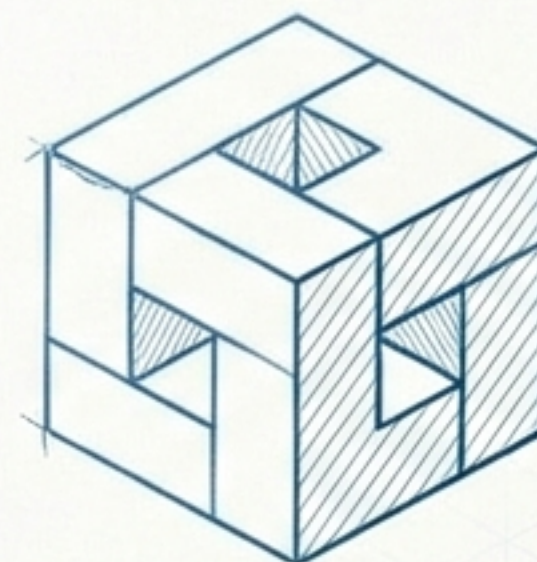
Entrada

Requisitos Funcionais
e Não Funcionais



Processo

Modelagem, Arquitetura,
Interfaces

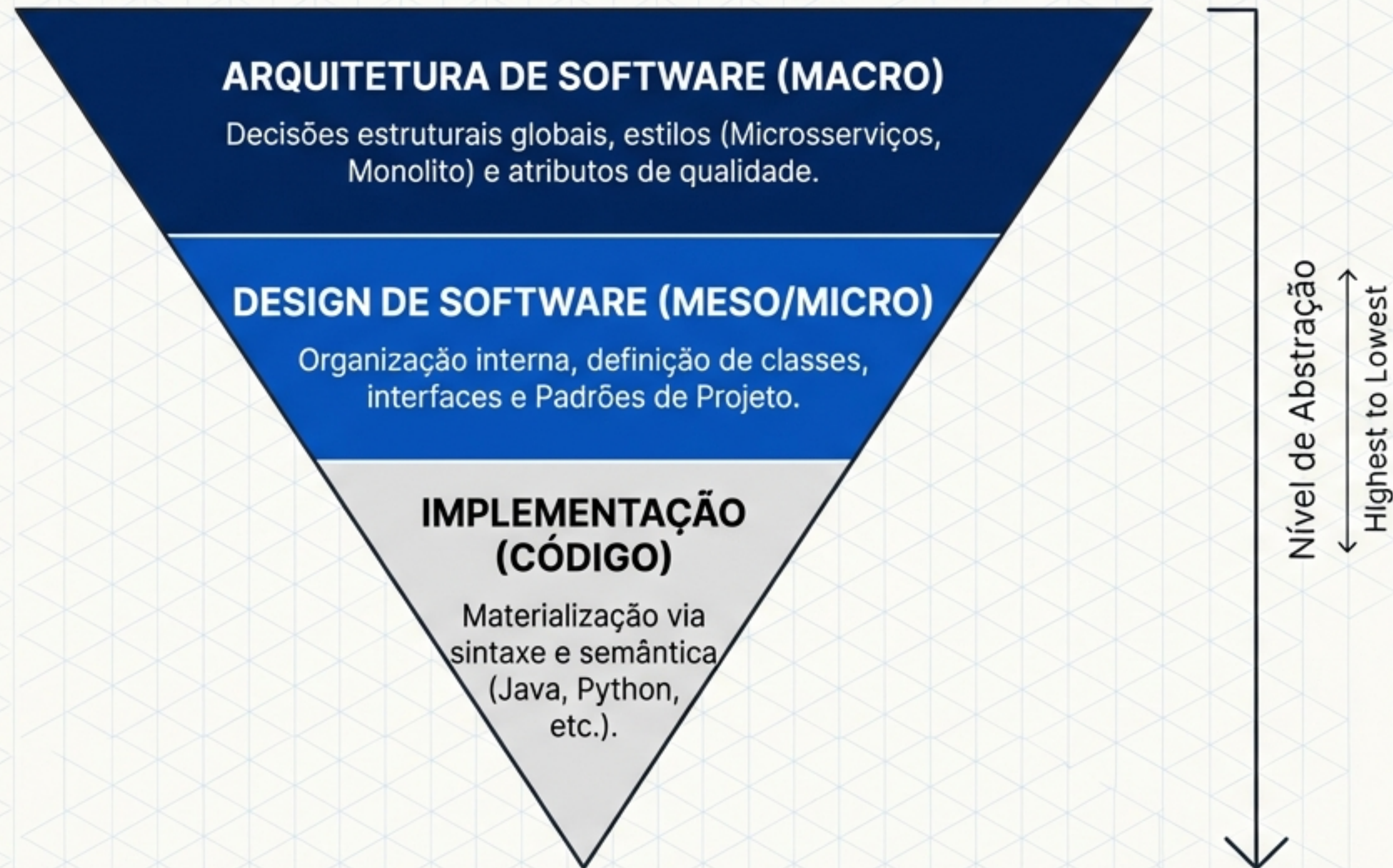


Saída

Solução Técnica e
Visão Holística

Fonte: Baseado na literatura clássica (Pressman e Sommerville).

A Hierarquia da Abstração



O Escopo do Design: Meso e Micro

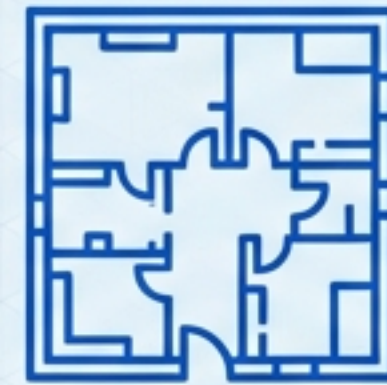
O Design atua dentro dos contornos estabelecidos pela arquitetura, definindo a **organização interna**.

A Arquitetura Define



- Haverá um serviço de persistência.
- Haverá uma API REST.

O Design Define

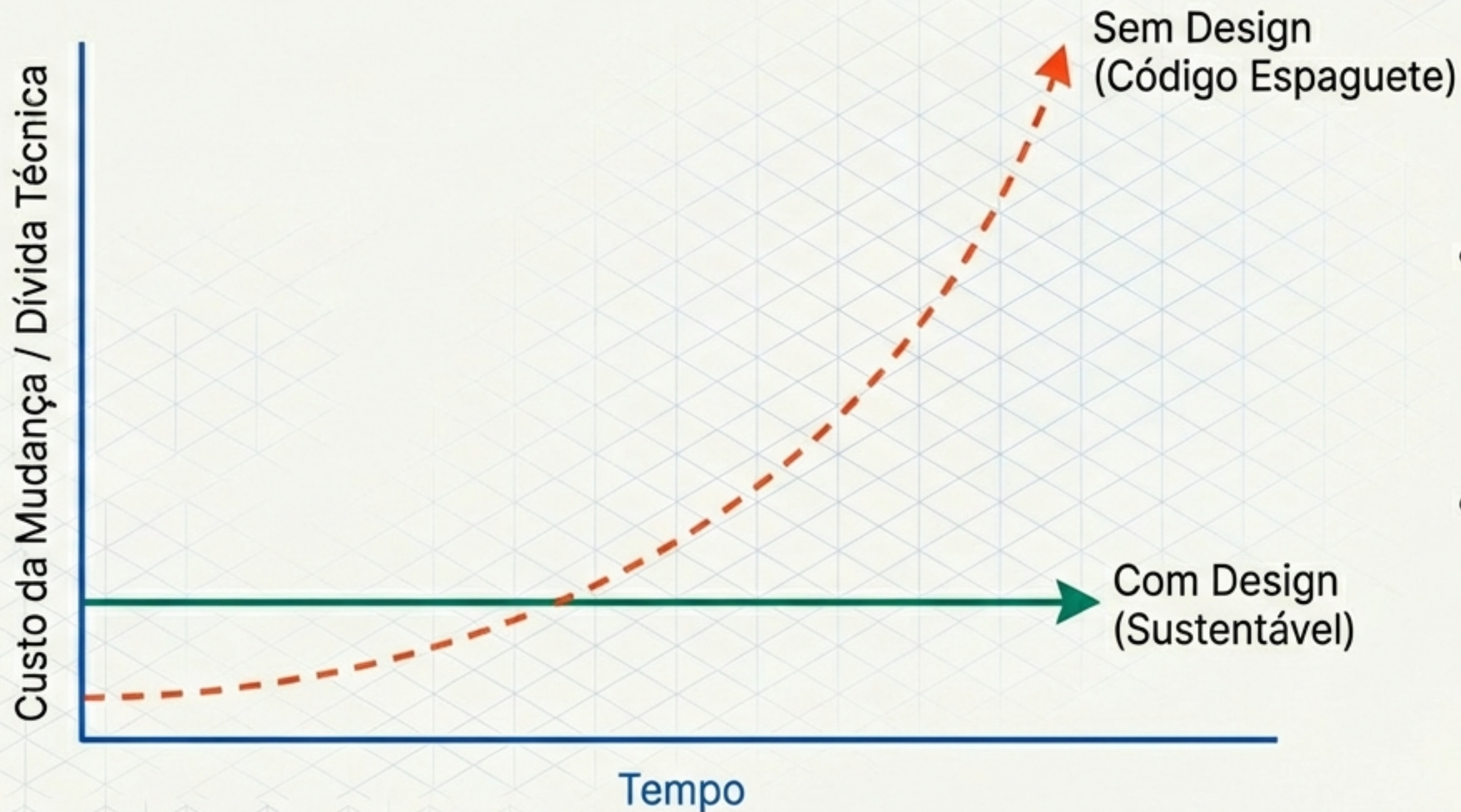


- Uso do padrão *Repository* para isolar SQL.
- Uso de *Injeção de Dependência* para controladores.

Conceito Chave: Delineamento das responsabilidades de cada entidade lógica.

A Relevância Estratégica

Por que investir tempo no design?



- **Problema:** A ausência de design planejado conduz à degradação arquitetural.
- **Consequência:** Dívida Técnica insustentável.

Os 4 Pilares da Qualidade Estrutural



1. Manutenibilidade

Capacidade de localizar e corrigir defeitos sem gerar efeitos colaterais (*side effects*).



2. Extensibilidade

Facilidade em acomodar novos requisitos sem reescrever estruturas (Princípio Aberto/Fechado).



3. Testabilidade

Componentes isolados que permitem testes unitários e de integração automatizados.



4. Reusabilidade

Componentes ortogonais que não dependem implicitamente de outros.

As Duas Métricas Soberanas

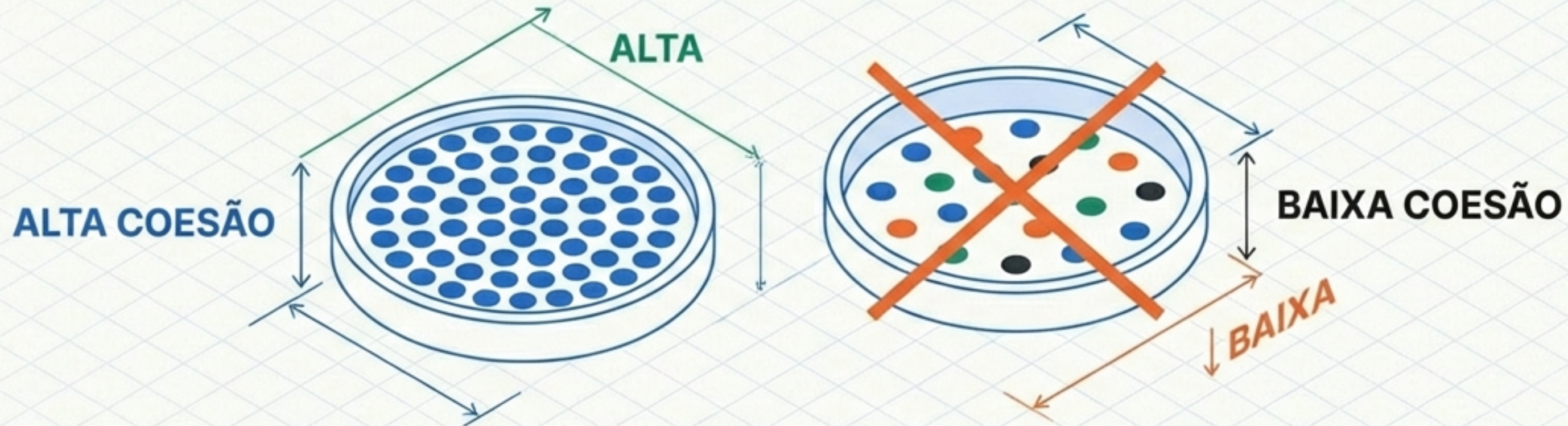
O objetivo central de todo projetista de software.



A excelência em design é balizada pelo equilíbrio dessas duas forças estruturais intrínsecas.

Coesão (Alta)

Mede o grau em que os elementos internos de um módulo pertencem uns aos outros.

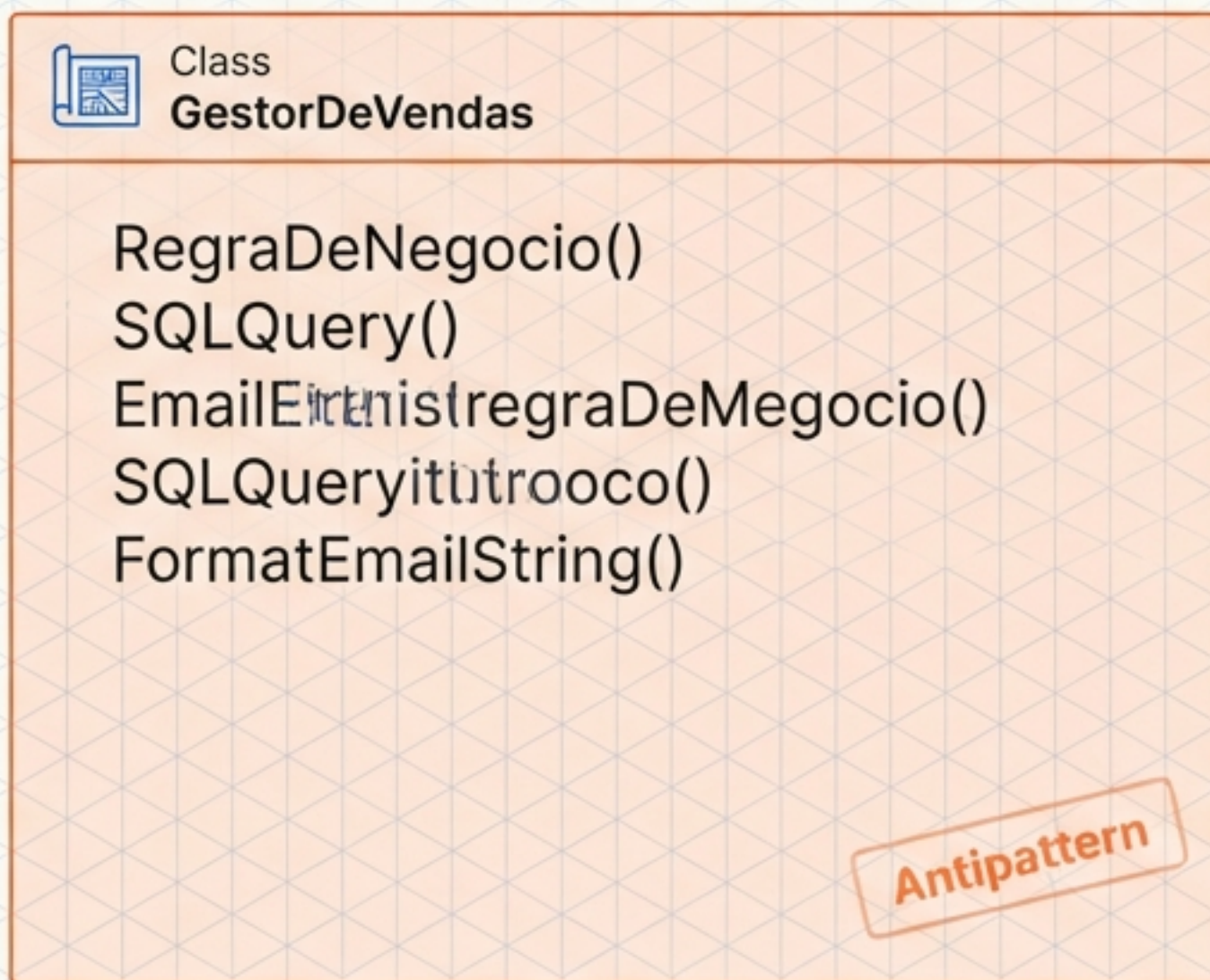


Base do **Single Responsibility Principle (SRP)**.

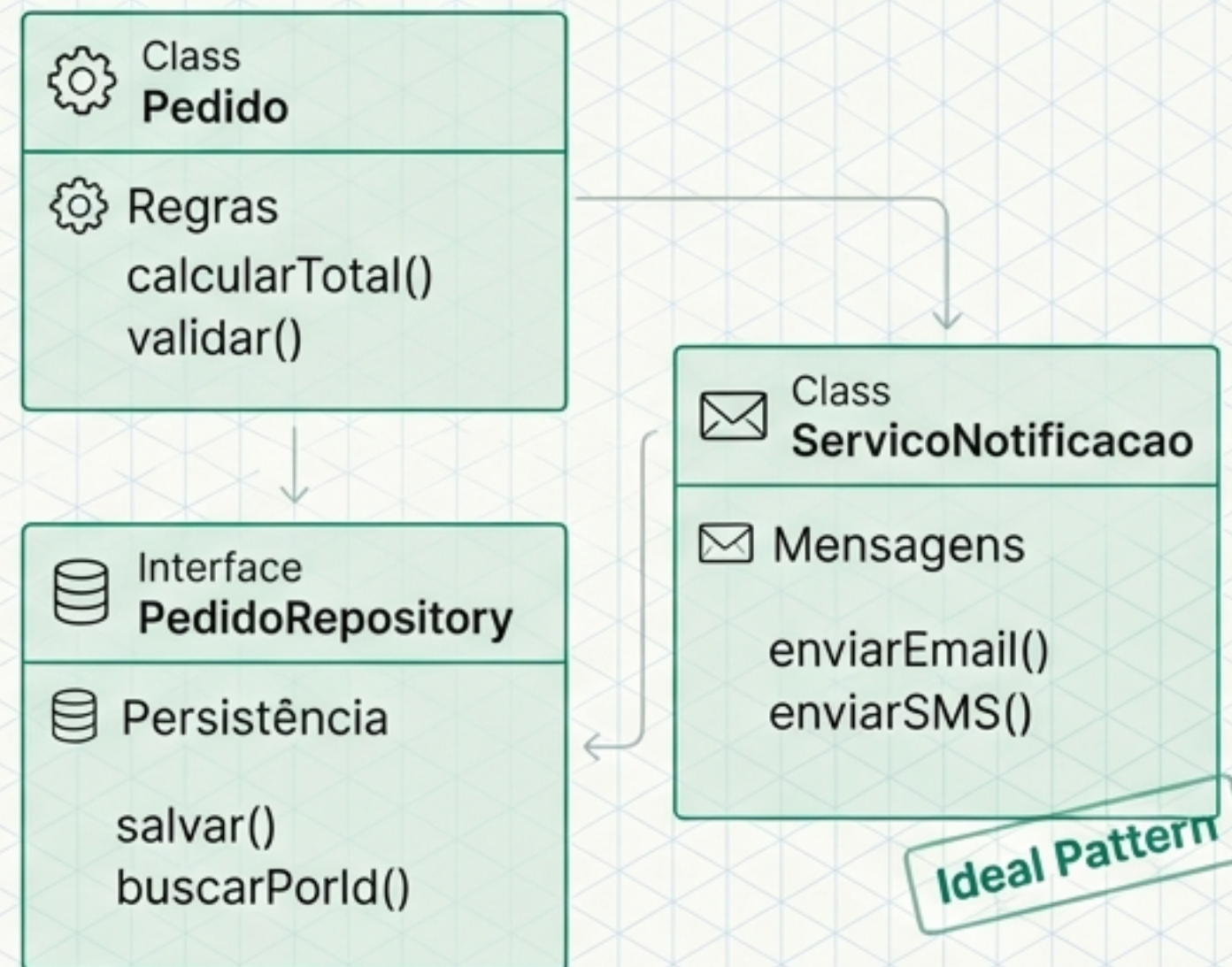
Uma classe altamente coesa possui uma única responsabilidade bem definida, encapsulando tudo o que é necessário para cumpri-la e nada além disso.

Estudo de Caso: Coesão

Antipadrão: Baixa Coesão



Padrão Ideal: Alta Coesão



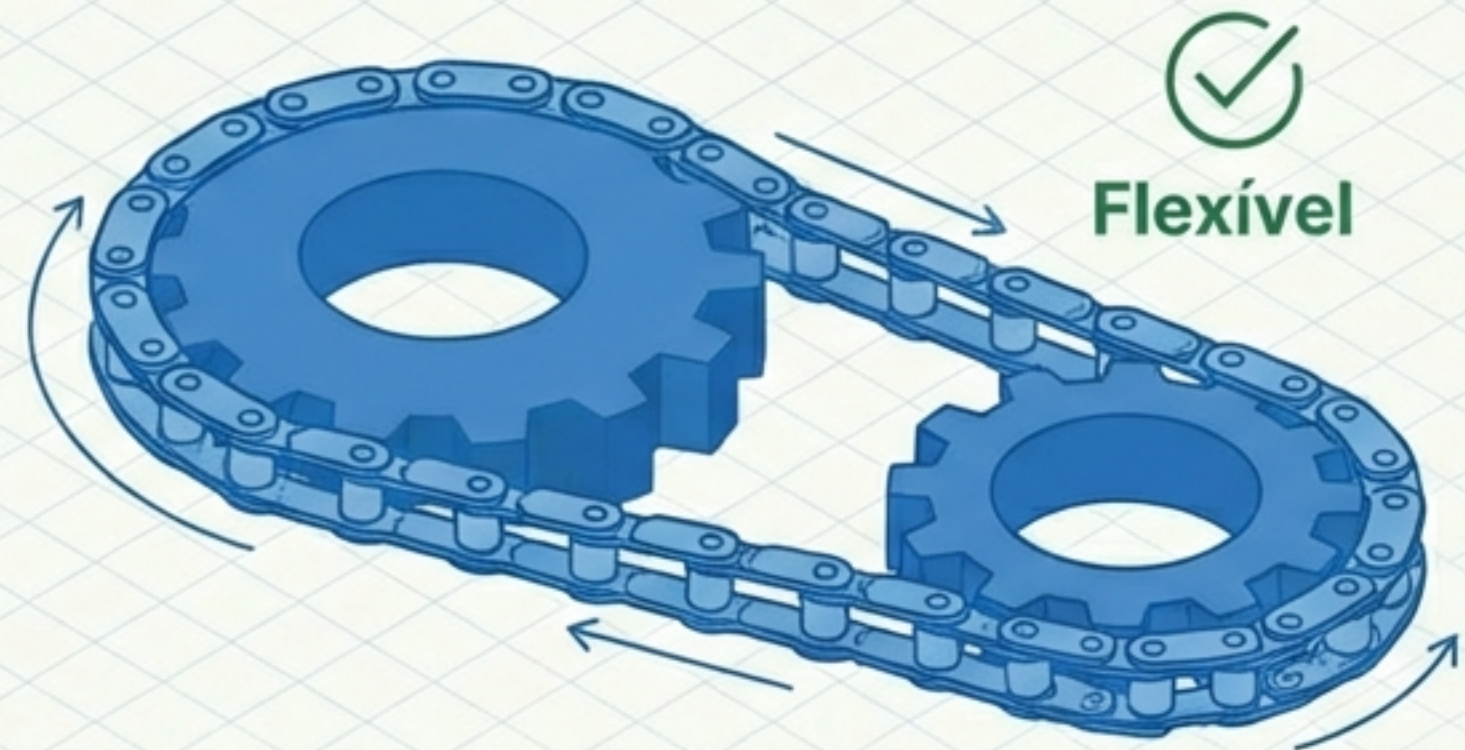
Acoplamento (Baixo)

Medida de interdependência entre os módulos de um sistema.

Diagram A (Bad)



Diagram B (Good)



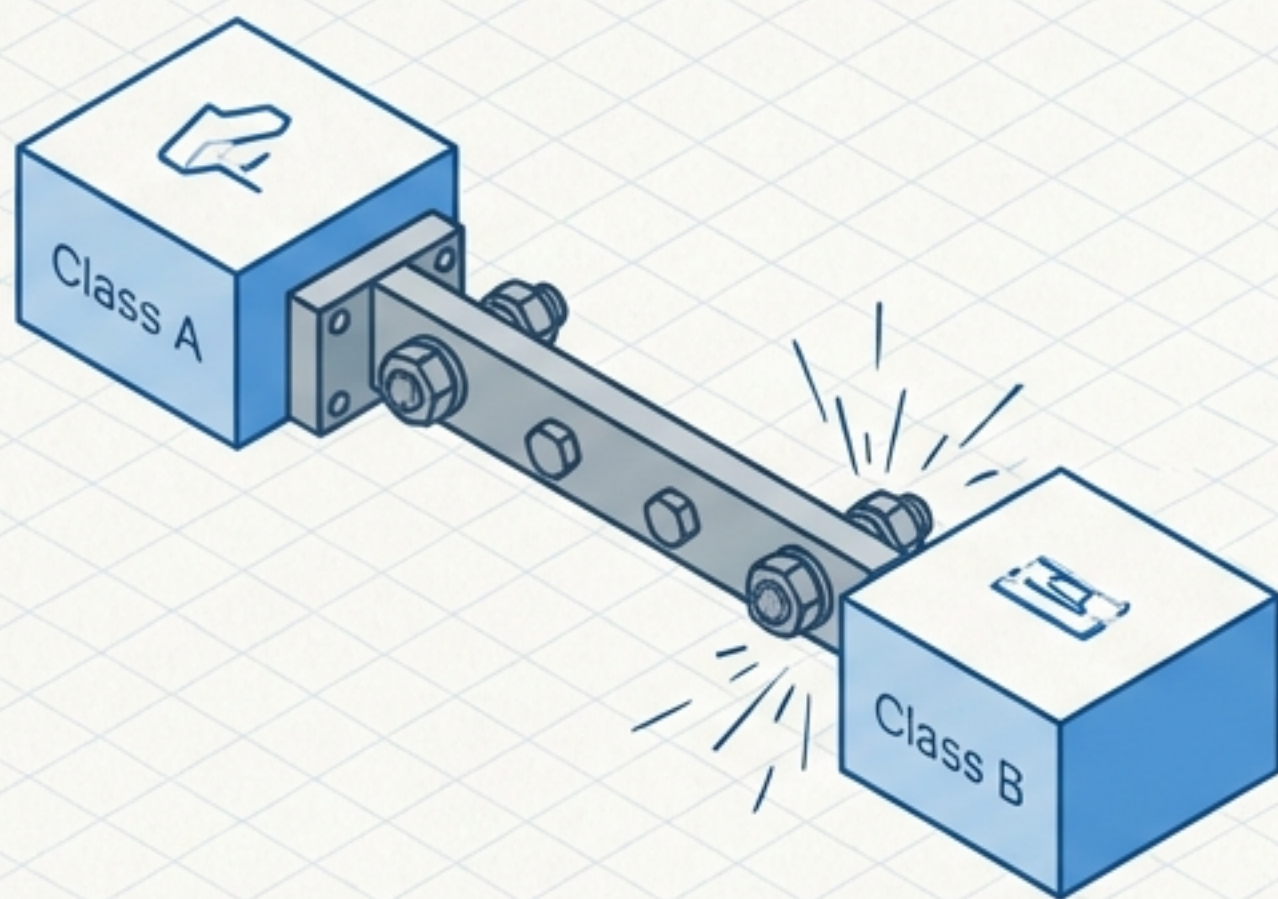
Acoplamento elevado significa que uma alteração em um módulo causa modificações em cascata (efeito dominó).



Módulos tão autônomos quanto possível.

Estudo de Caso: Acoplamento

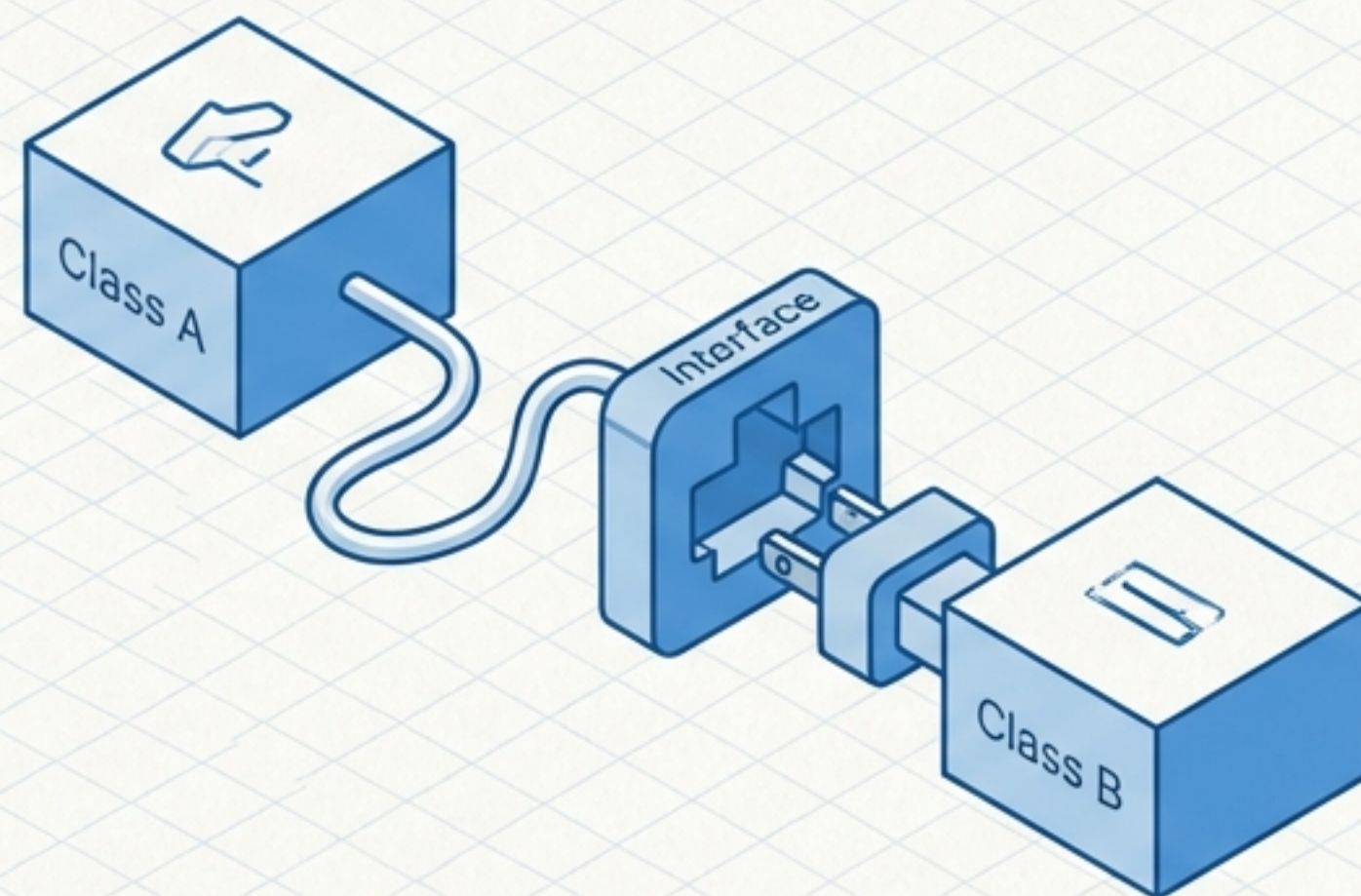
Antipadrão: Acoplamento Patológico



A instancia B diretamente
(**new B()**). Se B muda, **A** quebra.

Antipattern

Padrão Ideal: Baixo Acoplamento



Comunicação via Contratos.
Uso de **Injeção de Dependência**.

Ideal Pattern

O Design no Ciclo de Vida (SDLC)

Cascata / Waterfall



Ágil / Agile



O Ágil é design emergente, mas requer fundações.
Padrões transversais devem preceder a codificação.

O Manifesto do Engenheiro

O domínio do Design de Software é o que separa o codificador operacional do engenheiro de software pleno.



Construindo para o Futuro



A manipulação consciente da coesão e do acoplamento é o primeiro passo para a construção de sistemas robustos e escaláveis.

Fim da Aula 1